

# CompanyGraph – Domain- Driven Design als Seiten, die sich prüfen lassen

Ein Bounded Context, seine Sprache, seine Aggregate und Events als Seiten, über die innerhalb seiner Grenze eine Prüfung wacht.

BERUHT AUF `concept Pack`

Robert Blust · Software Engineer & Architect

# Die Methode, die Sie kennen

Das Pack schreibt die Begriffe von Domain-Driven Design als Seiten auf, statt eigene zu erfinden.

|          |                                                |
|----------|------------------------------------------------|
| EVANS    | <a href="#">Domain-Driven Design Reference</a> |
| VERNON   | <a href="#">Domain-Driven Design Distilled</a> |
| DDD CREW | <a href="#">Bounded Context Canvas</a>         |
| DDD CREW | <a href="#">Aggregate Design Canvas</a>        |
| DDD CREW | <a href="#">Context Mapping</a>                |
| JACKSON  | <a href="#">The Essence of Software</a>        |
| CUCUMBER | <a href="#">Gherkin reference</a>              |

BERUHT AUF concept Pack

# Bounded Context

Die Grenze, innerhalb derer ein Modell und eine Sprache gelten, als Seite: wofür der Kontext zuständig ist, wie er klassifiziert ist und welche Domäne er realisiert.

BERUHT AUF

bounded-context Resolution

model/bounded-contexts/resolution/resolution.md

```
---
id: 01a0faaa-5d02-71db-bcdb-f672c4324bee
source: Local
classification: core
realizes:
  - Core
decisions:
  - A reference resolves by its declared type, never by name alone
---

# Resolution
> Turns the names a page writes into the edges of the graph, by the type each schema declares and the owner a row names. Whether a page has the shape its schema asks for is left to the checks.

### Responsibilities
- Find the entity a written name means, looking only among the type the schema declares for that field or column
- Look for an owned entity only inside the owner its row names, so two owners can each have a page of the same name
...
```

# Seine Sprache

Concept Designs: die Begriffe aus der Sprache des Kontexts, jeder eine Entity oder ein Value Object, und jeder gehört seinem Kontext.

BERUHT AUF

concept-design Entity

concept-design Edge

concept-design Scope

concept-design Declaration

concept-design Graph

model/bounded-contexts/resolution/concept-designs/

|                |                    |
|----------------|--------------------|
| declaration.md | kind: value object |
| edge.md        | kind: value object |
| entity.md      | kind: value object |
| graph.md       | kind: entity       |
| scope.md       | kind: value object |

# Aggregat und Event

Ein Aggregat hält eine Gruppe von Concept Designs als Einheit konsistent. Ein Domain Event ist ein Typ, weil andere Kontexte darauf verweisen; ein Command ist eine Zeile, weil nichts ausserhalb seines Aggregats darauf verweist.

BERUHT AUF

aggregate Graph

domain-event Graph drawn

model/bounded-contexts/resolution/aggregates/graph.md

```

---
...
root: Graph
members:
  - Edge
  - Scope
...
# Graph
...
### Invariants
| Label | Invariant |
| --- | --- |
| INV-R1 | Every edge runs between two entities of the graph; a name that resolves to
nothing stops the read, and no edge points nowhere. |
...
### Handled commands
| Command | Emits | When | Description |
| --- | --- | --- | --- |
| Read an instance | | | Reads every page against the schemas the instance vendors and
draws the graph, or refuses at the first name that resolves to nothing |
...

```

# Context Map

Eine Beziehung zwischen Kontexten ist eine Zeile im abhängigen Kontext; eine symmetrische wird nur einmal geschrieben, bei einem der beiden Kontexte.

BERUHT AUF

bounded-context Checking

model/bounded-contexts/checking/checking.md

```
...  
### Relationships  
| Context | Pattern |  
| --- | --- |  
| Resolution | conformist |  
| Vendoring | shared kernel |  
...
```

# Feature Design

Wie ein Feature über die Kontexte hinweg gebaut wird, die es berührt: ein Operational Principle, dann Szenarien in Gherkin.

BERUHT AUF

```
feature-design A name that names nothing is reported where it is  
written
```

model/feature-designs/a-name-that-names-nothing-is-reported-where-it-is-written.md

```
---  
id: 01a10031-6925-7b3c-8f04-87b19b4f6103  
source: Local  
refines: Checks an instance runs  
contexts:  
  - Resolution  
...  
# A name that names nothing is reported where it is written  
...  
### Operational principle  
When a page names an entity that does not exist under the type its field declares, the  
  check fails on that page and says which type it searched, before the change can land.  
  
### Scenarios  
#### SC-C1: A name that exists nowhere  
...
```

# Wo das Pack abweicht

Wo das Pack anders vorgeht als seine Quellen, sagt sein README das.

BERUHT AUF

`decision A reference resolves by its declared type, never by name alone`

Strategische Klassifizierung

beim Bounded Context, wie der Bounded Context Canvas sie vornimmt, nicht bei der Subdomäne von Evans, sodass der Typ `domain` aus dem `core` unberührt bleibt

Ein Domain Event und ein Command

ein Event als Typ, weil andere Kontexte darauf verweisen; ein Command als Zeile, weil nichts ausserhalb seines Aggregats darauf verweist

Ein Architekturentscheid

kein eigener Typ, sondern der Entscheid aus dem `core`, auf den die Typen des Packs verweisen

Der Typ eines Werts

Term für einen Begriff des Kontexts, ein Verweis; Type für einen einfachen Typ, der auf nichts verweist

# Wie das Pack an den core anschliesst

Das Pack schliesst an die Typen aus dem core an: Ein Bounded Context realisiert eine Domäne, ein Concept Design verfeinert ein Konzept und ein Feature Design verfeinert ein Feature. Jede Kante verläuft nur in eine Richtung und ist optional.

BERUHT AUF

domain Core

feature Checks an instance runs

EBENE 1 · SOFTWARE

**Bounded Context · Concept Design · Feature Design**

↓ realisiert · verfeinert · verfeinert ↓

EBENE 0

**Core**

Domäne · Konzept · Feature

# Was die Prüfung durchsetzt

Ein Name wird nur innerhalb seines Typs gesucht und, bei einem Typ mit Besitzer, nur innerhalb seines Besitzers. Ein Name, der nichts benennt, wird dort als Fehler gemeldet, wo er steht.

BERUHT AUF `concept-design Entity · Publishing`

`concept-design Entity · Resolution`

`concept-design Entity · Serving`

`concept Check`

```
$ companygraph check
```

```
✗ 1 problem in model/ against meta/core, meta/software, meta/organization, meta/landscape/  
at core 0.65.0
```

```
model/bounded-contexts/resolution/concept-designs/edge.md: `Concept` in "### Relations" is  
declared `ref → concept-design` and says "Declarations", which names no entity in model/;  
a declared reference must resolve, and the concept-design it matches here is  
"Declaration" (R4)
```

```
not checked: every `### Writing rules` in every schema – that is the agent pass, R0
```

```
✓ 378 Markdown files in the one form
```

# Auf später verschoben

Was dem Pack noch fehlt, nennt sein README, statt es zu verbergen.

|        |                                                        |
|--------|--------------------------------------------------------|
| SPÄTER | Services, Repositories, Factories und Module           |
| SPÄTER | System, Container und Komponente aus C4                |
| SPÄTER | ein Typ für eine Beziehung zwischen Kontexten          |
| SPÄTER | Commands als Typ                                       |
| SPÄTER | ein Typ für Subdomänen                                 |
| SPÄTER | ein Feld für die Ebene                                 |
| SPÄTER | Read Models und Policies als Typen, aus Event Modeling |

BERUHT AUF `concept Pack`

# Ihre Sprache, innerhalb ihrer Grenze geprüft

Die eigenen Begriffe von Domain-Driven Design, geschrieben als Seiten, über die eine Prüfung wacht. Das Pack weicht nur dort von seinen Quellen ab, wo eine Seite es braucht.

BERUHT AUF **concept Pack**

EBENE 1 · SOFTWARE **Bounded Context · Concept Design · Feature Design**

↓ realisiert · verfeinert · verfeinert ↓

EBENE 0 **Core**  
Domäne · Konzept · Feature